Messaging and streaming

Choose and operate queues and logs: RabbitMQ and Kafka, delivery guarantees, consumer groups, lag, retention, schemas and managed cloud services.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Queues, logs and pub/sub: choosing a model (12 min)
2. Delivery guarantees, ordering and idempotency (13 min)
3. RabbitMQ: exchanges, queues and routing (13 min)
4. RabbitMQ: acknowledgements, prefetch and dead letters (13 min)
5. Kafka: topics, partitions and brokers (13 min)
6. Kafka: producers, consumers and consumer groups (14 min)
7. Kafka: replication, ISR and durability (13 min)
8. Kafka: retention, compaction and storage (12 min)
9. Kafka in operation: lag, rebalances and capacity (14 min)
10. Managed messaging on the cloud (13 min)
11. Schemas, contracts and evolving messages (13 min)
12. Debugging messaging incidents (14 min)
Cheat sheet

Lesson 1. Queues, logs and pub/sub: choosing a model

12 minutes. Compare work queues, publish-subscribe topics and append-only logs, and choose the right messaging model for a job.

Messaging is how services talk without waiting for each other. The three common shapes behave very differently under load, failure and change, so the first decision is the model, not the product.

What you will learn

- Work queues, publish-subscribe and logs
- What each model keeps and for how long
- Questions that decide the model
- Where messaging helps and where it hurts

Three shapes

A **work queue** holds jobs. Each message goes to exactly one worker and is removed when handled. Use it to spread work (send an email, resize an image). **Publish-subscribe** copies each message to every subscriber, so one event reaches billing, search and analytics. A **log** appends messages in order and keeps them for a set time or size. Consumers remember their own position (offset), so they can read at different speeds, join later and replay.

	Queue	Pub/sub topic	Log
Delivery	One consumer per message	Every subscriber	Every consumer group, by position
After reading	Message removed	Usually removed	Kept until retention ends
Replay	No	No	Yes
Typical tools	RabbitMQ, SQS	SNS, Cloud Pub/Sub	Kafka, Kinesis, Event Hubs

Why use messaging

It decouples senders from receivers in time (the receiver can be down), in speed (a buffer absorbs bursts) and in knowledge (the sender need not know who reads). The price is delay, more moving parts and new failure modes: duplicates, reordering, and a backlog that hides a failing consumer.

Diagram: Three messaging shapes

Part	What it does	What goes wrong
Producer	Sends messages without waiting for the reader.	If it cannot reach the broker it needs a retry or buffer.
Work queue	One message goes to one worker, then is removed.	A growing queue means workers are too slow or failing.
Pub/sub topic	Copies each message to every subscriber.	A slow subscriber can lose messages if there is no buffer.
Log with offsets	Keeps ordered messages; readers track their position.	Retention too short means a late reader misses data.
Consumers	Process messages at their own pace.	A consumer that fails silently lets the backlog grow.

Flow: Producer to Work queue / Producer to Pub/sub topic / Producer to Log with offsets / Work queue to Consumers / Pub/sub topic to Consumers / Log with offsets to Consumers

MARGA by EknathaLabs marga.eknathalabs.com

Choosing

Ask: Does each message need handling once by one worker (queue)? Must several systems each see every message (topic or log)? Will anyone need to replay history, rebuild a view or fix a bug by reprocessing (log)? How many messages per second, and how long must they live? Small teams with modest volume are often well served by a managed queue; high-volume event streams and replay point to a log.

Incident: The data that could not be replayed

> **Incident:** *A bug in a consumer corrupted a week of derived data, and the source messages were already deleted from the queue.*

```
$ rabbitmqctl list_queues name messages
orders   0
$ psql -c "select count(*) from order_totals where total < 0"
18422
```

The consumer had acked every message, so the broker kept nothing. The team moved the stream to a log with seven days of retention, fixed the consumer, and reprocessed from the offset before the bug started.

Common mistakes

- Choosing a product before choosing the model
- Using a queue when many systems need the same events
- Treating a message broker as a database
- Ignoring what happens when the consumer is down for hours

Check yourself

Which model lets a new consumer replay old messages?

Answer: A log with retention

In a work queue, how many consumers get each message?

Answer: One

What does pub/sub deliver?

Answer: A copy to every subscriber

Why does messaging add failure modes?

Answer: Duplicates, reordering and hidden backlogs

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Delivery guarantees, ordering and idempotency

13 minutes. Understand at-most-once, at-least-once and effectively-once delivery, when ordering holds, and how idempotent consumers make duplicates harmless.

The hardest promises in messaging are about what happens on failure. A sender cannot know whether a message arrived if the acknowledgement was lost, so it must choose between sending again and risking a duplicate, or not sending and risking a loss.

What you will learn

- At-most-once, at-least-once and exactly-once
- Where ordering is guaranteed and where it is not
- Idempotent consumers and idempotency keys
- The outbox pattern for sending messages reliably

The three promises

- **At-most-once:** send and forget. No duplicates, but a lost message is gone.
- **At-least-once:** retry until acknowledged. No loss, but duplicates happen after a lost acknowledgement or a consumer crash.
- **Exactly-once:** each message has one effect. Brokers can offer it within their own system (Kafka transactions), but across a database and a broker it is achieved by making the **effect** idempotent.

Most production systems run at-least-once and make handlers safe to repeat.

Ordering

Messages are ordered only within a single queue or **partition**. Two consumers on one queue may finish out of order, and retries reorder messages. To keep related messages in order, send them with the same **key** (an order id) so they land in the same partition, and process each partition sequentially.

Idempotent consumers

```
-- processed_messages(message_id primary key)
begin;
insert into processed_messages(message_id) values ('evt-7f3a')
  on conflict do nothing;
-- if 0 rows were inserted, the message was already handled: skip
update accounts set balance = balance - 20 where id = 42;
commit;
```

Store the message id and the effect in one transaction. Or use naturally idempotent operations: `set status = 'shipped'` is safe to repeat, `balance = balance - 20` is not without a guard.

The outbox pattern

Writing to the database and publishing to a broker are two actions that can half-fail. Write the event to an **outbox table** in the same transaction as the data change; a separate relay reads the table and publishes, retrying until the broker confirms.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: Safe sending and receiving

Part	What it does	What goes wrong
Service	Changes data and writes an outbox row in one transaction.	Publishing directly can lose the event if the process dies between steps.
Outbox table	Holds events until they are published.	Without cleanup it grows forever.
Relay	Publishes outbox rows to the broker and retries.	Retries cause duplicates, which the consumer must handle.
Broker	Delivers at least once to consumers.	An acknowledgement lost on the way causes redelivery.
Idempotent consumer	Checks the message id before applying the effect.	Without the check a redelivery repeats the effect.

Flow: Service to Outbox table / Outbox table to Relay / Relay to Broker / Broker to Idempotent consumer

Incident: Charged twice

Incident: *A payment consumer crashed after charging but before it acknowledged the message, so the broker redelivered it.*

```
payments.log: charge ok order=4411 amount=49.00
payments.log: consumer restarted
payments.log: charge ok order=4411 amount=49.00
support: 1 customer charged twice
```

The consumer had no record of what it had handled. The team added an idempotency key (the order id) to the payment call and a processed table, then refunded the duplicates found by a one-off query.

Common mistakes

- Assuming exactly-once because the broker advertises it
- Acknowledging a message before the work is saved
- Relying on global ordering across partitions
- Publishing straight after a database commit with no outbox

Check yourself

What can happen with at-least-once delivery?

Answer: Duplicates

How is ordering usually guaranteed?

Answer: Within a partition or queue, using a key

What makes a consumer idempotent?

Answer: Handling a repeat without repeating the effect

What problem does the outbox pattern solve?

Answer: Doing a database change and a publish reliably together

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. RabbitMQ: exchanges, queues and routing

13 minutes. Understand RabbitMQ's model of producers, exchanges, bindings and queues, and choose direct, topic, fanout and headers exchanges.

RabbitMQ is a widely used message broker built on AMQP. Its power is in routing: the producer does not know which queues exist, only the exchange and a routing key. Learning that one layer of indirection makes the rest of RabbitMQ easy to reason about.

What you will learn

- Producers, exchanges, bindings and queues
- Direct, topic, fanout and headers exchanges
- Durability and persistence
- Virtual hosts, users and the management interface

The model

A **producer** publishes a message to an **exchange** with a **routing key**. The exchange applies its rules to **bindings** (links from the exchange to queues) and copies the message into each matching **queue**. **Consumers** read from queues.

Exchange	Routes by
direct	Exact match of routing key to binding key
topic	Pattern match: `orders.*.eu`, `#` for everything
fanout	Every bound queue, ignoring the key
headers	Message header values

```
$ rabbitmqadmin declare exchange name=events type=topic durable=true
$ rabbitmqadmin declare queue name=billing durable=true
$ rabbitmqadmin declare binding source=events destination=billing routing_key='order.paid.#'
$ rabbitmqadmin publish exchange=events routing_key=order.paid.eu payload='{"id":4411}'
```

Durability

Messages survive a broker restart only if the **queue is durable** and the **message is persistent**. For stronger safety use **quorum queues**, which replicate across nodes using a consensus protocol and are the recommended type for important data. Classic non-replicated queues lose messages if their node is lost.

Virtual hosts and access

A **vhost** is an isolated namespace of exchanges, queues and permissions. Use one per environment or application. Give each service its own user with permissions only on its vhost. The management UI (port 15672) and `rabbitmqctl list_queues` show message counts, consumers and rates.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How RabbitMQ routes a message

Part	What it does	What goes wrong
Producer	Publishes to an exchange with a routing key.	Publishing to a missing exchange fails or the message is dropped.
Exchange	Applies routing rules to decide which queues get a copy.	A wrong binding silently drops messages that match nothing.
Billing queue	Bound with order.paid.# and read by billing.	Not durable means messages vanish on restart.
Search queue	Bound to the same key for another consumer.	A slow consumer fills memory and disk.
Consumers	Read, process and acknowledge messages.	No consumer means the queue only grows.

Flow: Producer to Exchange / Exchange to Billing queue / Exchange to Search queue / Billing queue to Consumers

Incident: The messages that matched nothing

Incident: *A new routing key had a typo and an exchange dropped events with no error.*

```
$ rabbitmqctl list_exchanges name type
events topic
management: events unroutable_dropped 3210
binding: events -> billing  (order.paid.#)
```

The producer published order.payed.eu, which matched no binding, so the exchange dropped the messages. The team turned on the mandatory flag and an alternate exchange for unroutable messages, and added an alert on the dropped count.

Common mistakes

- Publishing to the default exchange with a queue name everywhere
- Non-durable queues for important work
- One shared user and vhost for every application
- No alert for unroutable messages

Check yourself

Where do producers publish in RabbitMQ?

Answer: To an exchange

Which exchange type matches patterns like order.*?

Answer: topic

What makes a message survive a restart?

Answer: A durable queue and a persistent message

What are quorum queues for?

Answer: Replicated, safer queues for important data

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. RabbitMQ: acknowledgements, prefetch and dead letters

13 minutes. Handle failures in RabbitMQ with acknowledgements, prefetch limits, retries, dead-letter exchanges and monitoring of queue depth.

Most RabbitMQ incidents are about consumers: not acknowledging, taking too much, or retrying a bad message forever. Three settings and one pattern, acknowledgements, prefetch, TTL and dead-lettering, prevent most of them.

What you will learn

- Manual acknowledgements and publisher confirms
- Prefetch and fair dispatch
- Dead-letter exchanges and retry delays
- Reading ready, unacked and consumer numbers

Acknowledgements

With **manual acks** the broker keeps a message as **unacked** until the consumer confirms it. If the connection drops, the message returns to the queue and is delivered again (at-least-once). Acknowledge only after the work is saved. On the producer side, **publisher confirms** tell the producer the broker has safely taken the message.

Prefetch

```
channel.basic_qos(prefetch_count=20)     # at most 20 unacked per consumer
channel.basic_consume(queue='billing', on_message_callback=handle, auto_ack=False)
```

Without a limit one consumer can receive thousands of messages and sit on them while others are idle. A sensible prefetch (10 to 100 for quick tasks, 1 for slow ones) spreads work fairly and bounds memory.

Dead letters and retries

```
$ rabbitmqadmin declare queue name=billing durable=true \
    arguments='{"x-dead-letter-exchange":"dlx","x-message-ttl":600000}'
```

When a message is rejected without requeue, expires, or the queue overflows, the broker republishes it to the **dead-letter exchange**. A common design: a `retry` queue with a TTL (say 30 s) whose dead-letter target is the original queue, giving a delayed retry; after N attempts (counted in a header) the message goes to a `parked` queue for humans.

What to watch

Metric	Meaning	Worry when
Ready	Waiting for a consumer	Growing steadily
Unacked	Delivered, not yet acknowledged	Large and stuck
Consumers	Connected consumers	Zero
Publish vs ack rate	In versus out	Publish far above ack

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Failure handling in RabbitMQ

Part	What it does	What goes wrong
Main queue	Delivers up to prefetch messages to each consumer.	Without prefetch one consumer can hoard messages.
Consumer	Processes then acknowledges, or rejects without requeue.	Acking too early loses work if it crashes next.
Retry queue	Holds a failed message for a TTL, then returns it.	Unlimited retries loop forever on a poison message.
Parked queue	Keeps messages that failed N times for a human.	Nobody watching it means silent data loss.
Alerts	Watch ready, unacked and parked counts.	An alert on only queue length misses a stuck consumer.

Flow: Main queue to Consumer / Consumer to Retry queue / Retry queue to Main queue / Retry queue to Parked queue / Parked queue to Alerts

Incident: The poison message

Incident: *One malformed message crashed its consumer, was requeued at once and blocked the queue.*

```
$ rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
orders  52113  1  1
consumer log: JSONDecodeError at offset 0 (restart 480)
```

The bad message was rejected with requeue true, so it came back first every time. The fix was reject without requeue to a dead-letter queue, plus a parked queue alert, so the 52,000 good messages could be processed.

Common mistakes

- Using auto-ack for work that must not be lost
- Rejecting with requeue true for messages that can never succeed
- No prefetch limit
- A dead-letter queue with no owner or alert

Check yourself

What does prefetch limit?

Answer: Unacknowledged messages per consumer

What happens if a consumer dies before acking?

Answer: The message is redelivered

Where do rejected messages go when a DLX is set?

Answer: To the dead-letter exchange

Which sign suggests a stuck consumer?

Answer: Large unacked count and no progress

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Kafka: topics, partitions and brokers

13 minutes. Learn Kafka's model of topics split into partitions, brokers that store them, offsets and keys, and how partitioning shapes ordering and scale.

Kafka is a distributed commit log. Producers append records, consumers read at their own pace, and the data stays for a chosen time. Everything about scale and ordering comes from one idea, the partition, so it is the first thing to understand.

What you will learn

- Topics, partitions, offsets and records
- Brokers, replicas and the controller
- Keys and how they choose a partition
- Choosing a partition count

Topics and partitions

A **topic** is a named stream. It is divided into **partitions**; each partition is an ordered, append-only file split into segments on disk. Each record in a partition gets an **offset**: 0, 1, 2 and so on. There is no global order across partitions, only within one.

```
$ kafka-topics.sh --bootstrap-server b1:9092 --create --topic orders --partitions 6 --replication- \
    factor 3
$ kafka-topics.sh --bootstrap-server b1:9092 --describe --topic orders
Topic: orders  PartitionCount: 6  ReplicationFactor: 3
  Partition: 0  Leader: 1  Replicas: 1,2,3  Isr: 1,2,3
  Partition: 1  Leader: 2  Replicas: 2,3,1  Isr: 2,3,1
```

Brokers and replicas

A **broker** is a Kafka server. Each partition has one **leader** broker that handles reads and writes, and **followers** that copy it. The set of followers that are caught up is the **ISR** (in-sync replicas). Modern clusters run in **KRaft** mode, where Kafka brokers manage cluster metadata themselves with a Raft quorum instead of using ZooKeeper.

Keys

A producer may attach a **key**. Records with the same key hash to the same partition, so they stay in order. With no key, records are spread across partitions. Use the entity id (order id, user id) as the key when order per entity matters.

How many partitions

Partitions set the maximum parallelism of a **consumer group**: with 6 partitions at most 6 consumers read in parallel. More partitions give more throughput and more files, open handles and recovery time. Adding partitions later is possible but changes which partition a key maps to; reducing is not supported. Plan for the growth you expect, then add headroom.

Diagram: A Kafka topic across brokers

Part	What it does	What goes wrong
Producers	Append records, choosing a partition by key.	A hot key sends all load to one partition.
Broker 1	Leader for some partitions, follower for others.	Losing it moves leadership to followers in the ISR.
Broker 2	Holds replicas and leaders too.	A slow disk here slows its leader partitions.
Broker 3	Third copy for fault tolerance.	Two brokers down with replication 3 can stop writes.
Consumer group	Each partition is read by one consumer in the group.	More consumers than partitions leaves some idle.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Producers to Broker 1 / Producers to Broker 2 / Broker 1 to Broker 3 / Broker 2 to Consumer group

Incident: Three partitions, twenty consumers

Incident: *A service needed to scale to twenty consumers but the topic had only three partitions.*

```
$ kafka-consumer-groups.sh --bootstrap-server b1:9092 --describe --group invoicer
GROUP     TOPIC   PARTITION  LAG
invoicer  orders  0          1,401,000
invoicer  orders  1          1,380,000
invoicer  orders  2          1,352,000
17 members have no partition assigned
```

Adding consumers did nothing. The team created a new topic with 24 partitions, copied the old data and cut over producers, because adding partitions to the old topic would have broken key ordering for existing data.

Common mistakes

- Choosing too few partitions to save effort
- Using a low-cardinality key that makes one hot partition
- Expecting order across partitions
- Adding partitions without understanding the key mapping change

Check yourself

Where does Kafka guarantee order?

Answer: Within a partition

What does the ISR contain?

Answer: Replicas that are caught up with the leader

What limits parallel consumers in a group?

Answer: The number of partitions

Why choose a record key?

Answer: Records with the same key stay in order

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Kafka: producers, consumers and consumer groups

14 minutes. Configure producers for durability and throughput, consumers for safe offset commits, and understand how consumer groups share partitions.

Kafka clients carry much of the responsibility. A producer decides how durable a write must be; a consumer decides when its progress is saved. Wrong defaults in either quietly lose or repeat data.

What you will learn

- Producer acknowledgements, idempotence and batching
- The consumer poll loop and offset commits
- Consumer groups, assignment and rebalances
- Settings that reduce rebalance trouble

Producer settings

```
acks=all                    # wait for all in-sync replicas
enable.idempotence=true     # no duplicates from producer retries
retries=2147483647
delivery.timeout.ms=120000
linger.ms=10                # wait a little to build bigger batches
batch.size=65536
compression.type=lz4
```

`acks=all` with `min.insync.replicas=2` means a record is acknowledged only after at least two replicas have it. `enable.idempotence` lets the broker discard duplicates caused by retries. Batching and compression raise throughput at the cost of a few milliseconds.

The consumer loop

```
consumer = KafkaConsumer('orders', group_id='invoicer', enable_auto_commit=False)
for batch in consumer:
    handle(batch)            # do the work first
    consumer.commit()        # then record progress
```

Auto-commit can commit offsets for records you have not finished. Commit after processing for **at-least-once**; make `handle` idempotent so a replay after a crash is safe.

Consumer groups and rebalances

All consumers with the same `group.id` share a topic: a **group coordinator** broker assigns each partition to one member. When a member joins, leaves or stops sending heartbeats, the group **rebalances** and partitions move. During a classic rebalance, consumption pauses. Reduce the pain with `session.timeout.ms` and `max.poll.interval.ms` set to match real processing time, cooperative rebalancing, and **static membership** (`group.instance.id`) so a quick restart does not trigger a rebalance.

Diagram: A consumer group sharing partitions

Part	What it does	What goes wrong
Topic with 6 partitions	The data to be read.	A hot partition makes one consumer slower than the rest.
Group coordinator	Assigns partitions and tracks heartbeats.	If it is slow, rebalances take longer.
Consumer 1	Reads partitions 0 to 2.	If it stalls, the group sees lag on its partitions.
Consumer 2	Reads partitions 3 to 5.	A crash triggers a rebalance for the whole group.
Committed offsets	The saved position for each partition.	Committing early loses records; late repeats them.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Group coordinator to Consumer 1 / Group coordinator to Consumer 2 / Topic with 6 partitions to Consumer 1 / Topic with 6 partitions to Consumer 2 / Consumer 1 to Committed offsets

Incident: Committed before it was done

Incident: *Offsets were auto-committed while a batch was still processing, and a crash lost 800 records.*

```
$ kafka-consumer-groups.sh --describe --group invoicer
orders 0  CURRENT-OFFSET 5000  LOG-END-OFFSET 5000  LAG 0
$ psql -c "select count(*) from invoices"
4200
```

Lag showed zero, which looked healthy, but 800 records had been marked as read and never processed. The team moved to manual commits after the write and added a count check between topic and table.

Common mistakes

- Leaving enable.auto.commit on with slow processing
- acks=1 for data that must not be lost
- A poll interval shorter than the processing time, causing endless rebalances
- Running more consumers than partitions

Check yourself

When should a consumer commit offsets for at-least-once?

Answer: After processing

What does acks=all mean?

Answer: Wait for all in-sync replicas

What triggers a rebalance?

Answer: A member joining, leaving or timing out

What is static membership for?

Answer: Avoiding rebalances on quick restarts

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Kafka: replication, ISR and durability

13 minutes. Understand leader election, in-sync replicas, min.insync.replicas and unclean elections so you can reason about what Kafka loses when brokers fail.

Durability in Kafka is not one switch but a combination of replication factor, acknowledgement level and minimum in-sync replicas. Knowing how they interact tells you exactly what a given failure costs.

What you will learn

- Leaders, followers and the in-sync set
- replication.factor, min.insync.replicas and acks together
- Unclean leader election and its trade-off
- Rack awareness and safe maintenance

Leaders, followers and the ISR

Producers write to the partition **leader**; followers fetch from it. A follower that keeps up stays in the **ISR**. If the leader fails, the controller picks a new leader from the ISR, so no acknowledged record is lost. A follower that falls behind drops out of the ISR.

The three settings

Setting	Meaning	Common value
`replication.factor`	Copies of each partition	3
`min.insync.replicas`	Minimum ISR size to accept `acks=all` writes	2
producer `acks`	How many acknowledgements to wait for	`all`

With 3 copies, `min.insync.replicas=2` and `acks=all`, one broker can fail with no data loss and no downtime. If a second fails, the ISR drops below 2 and writes are refused (the partition favours safety over availability). With `acks=1` a leader crash before followers copy can lose acknowledged records.

Unclean election

If every ISR member is gone, the choices are to wait for one to return, or enable `unclean.leader.election.enable=true` and elect an out-of-sync replica. The second restores service quickly but **discards records** the replica never copied. Leave it off for data you cannot lose.

Failure domains and maintenance

Set `broker.rack` so replicas land in different racks or zones. For restarts use controlled shutdown, and roll one broker at a time, waiting until under-replicated partitions return to zero before moving on.

Diagram: What replication protects

Part	What it does	What goes wrong
Producer acks=all	Waits until enough replicas have the record.	acks=1 can lose a record if the leader dies at once.
Leader	Accepts writes and serves reads for the partition.	If it fails, the controller elects a new one from the ISR.
Follower in ISR	Caught up and eligible to become leader.	Falling behind removes it from the ISR.
Follower out of sync	Lagging copy, not safe to elect.	Electing it (unclean) loses the missing records.
Controller	Chooses leaders and tracks the ISR.	A controller election pauses leadership changes briefly.

Flow: Producer acks=all to Leader / Leader to Follower in ISR / Leader to Follower out of sync / Controller to Leader

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: Two brokers in one rack

Incident: *A rack lost power and took two of three replicas of several partitions.*

```
$ kafka-topics.sh --describe --under-min-isr-partitions
Topic: orders Partition: 3 Leader: 2 Replicas: 1,2,3 Isr: 2
producer: NotEnoughReplicasException: Messages are rejected since there are fewer in-sync replicas \
     than required
```

The producers stopped rather than lose data, which was the intended behaviour. After power returned the followers caught up. The team added broker.rack so replicas would spread across racks, and tested a rack failure in a game day.

Common mistakes

- Replication factor 1 on important topics
- acks=1 combined with a belief that data is safe
- Enabling unclean election without understanding the loss
- Restarting several brokers at once

Check yourself

With RF 3, min ISR 2 and acks=all, how many broker failures keep writes working?

Answer: One

What does unclean leader election risk?

Answer: Losing records the replica never copied

What does broker.rack help with?

Answer: Spreading replicas across failure domains

Before restarting the next broker, what should be zero?

Answer: Under-replicated partitions

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Kafka: retention, compaction and storage

12 minutes. Decide how long Kafka keeps data, when to use log compaction, and how segments, disks and tiered storage affect cost and recovery time.

Kafka's cost is mostly disk. Retention is the lever: how many days of history, or how many bytes, to keep. Compaction is a different mode that turns a topic into a table of latest values. Choosing well saves money and avoids a full disk.

What you will learn

- Time and size retention, and how segments are deleted
- Log compaction, keys and tombstones
- Estimating disk needs
- Tiered storage and long-term data

Deletion retention

```
$ kafka-configs.sh --bootstrap-server b1:9092 --alter --entity-type topics --entity-name orders \
    --add-config retention.ms=604800000,retention.bytes=107374182400
```

Kafka deletes data a **segment** at a time, not record by record. A segment is closed when it reaches `segment.bytes` or `segment.ms`; it becomes eligible for deletion when its newest record is older than `retention.ms`. The active segment is never deleted, so data can live longer than the setting suggests. `retention.bytes` applies per partition, not per topic.

Compaction

With `cleanup.policy=compact`, Kafka keeps at least the latest record for every key and removes older ones in the background. The topic becomes a changelog of current state: user profiles, configuration, the latest price per product. A record with a key and a null value is a **tombstone** that deletes that key after `delete.retention.ms`. Compaction does not remove records from the active segment or guarantee a time, and consumers can still see older duplicates until cleaning runs.

Estimating disk

Disk per broker is roughly: write rate × retention × replication factor ÷ number of brokers, plus 30 percent or more headroom. For 20 MB/s, 7 days and RF 3 across 6 brokers: 20 × 604,800 × 3 ÷ 6 ≈ 6 TB per broker before headroom.

Tiered storage

Newer Kafka versions can move old segments to cheaper object storage while keeping recent data on local disks, so you can keep long history without huge broker disks. Reads of old data are slower.

Diagram: How Kafka frees disk

Part	What it does	What goes wrong
Producers write	Records append to the active segment.	A bug that doubles traffic doubles disk growth.
Closed segments	A segment closes at its size or age limit.	The active segment is never deleted.
Retention check	Deletes segments older than retention.ms or beyond retention.bytes.	Size limit above the free disk allows a full disk.
Compaction	Keeps the latest record per key for compact topics.	Tombstones delete a key; do not use null for other things.
Disk alert	Fires when free space is low.	Alerting at 95 percent leaves no time to react.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Producers write to Closed segments / Closed segments to Retention check / Closed segments to Compaction / Closed segments to Disk alert

Incident: The full disk on Sunday

Incident: *A producer bug doubled traffic, and retention kept more than the disks could hold.*

```
$ df -h /var/lib/kafka
/dev/nvme1n1  4.0T  4.0T  0  100%
kafka: java.io.IOException: No space left on device
kafka: Shutting down because log directory failed
```

After they freed space by lowering retention on the noisy topic, brokers recovered. The team set retention from real rates, added a disk alert at 70 percent and a quota on the misbehaving producer.

Common mistakes

- Setting retention.bytes without noticing it is per partition
- Expecting compaction to run immediately
- No disk alert or producer quota
- Using a compacted topic for events that should all be kept

Check yourself

How does Kafka delete data under time retention?

Answer: Whole segments at a time

What does log compaction keep?

Answer: The latest record for each key

What is a tombstone?

Answer: A record with a key and null value that deletes the key

What is a safe disk planning step?

Answer: Add headroom for bugs and growth

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Kafka in operation: lag, rebalances and capacity

14 minutes. Monitor consumer lag, under-replicated partitions and request latency, size a cluster, and respond to common production problems.

A Kafka cluster that works on day one needs watching on day 100. The same few signals reveal almost every trouble: consumers behind, replicas out of sync, disks filling, and a controller that keeps changing.

What you will learn

- Consumer lag as records and as time
- Broker health metrics that matter
- A decision path for lag and rebalance incidents
- Capacity planning and safe rolling changes

Consumer lag

Lag is the difference between the newest offset in a partition and the consumer's committed offset.

```
$ kafka-consumer-groups.sh --bootstrap-server b1:9092 --describe --group invoicer
TOPIC   PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG    CONSUMER-ID
orders  0          1,200,000       1,200,040       40     c-1
orders  1          900,000         1,250,000       350000 c-2
```

One partition far behind points to a slow or stuck consumer or a hot key; all partitions behind points to a consumer group that is too slow overall or a producer burst. Convert lag to time: lag divided by the consumption rate. "12 minutes behind" is easier to alert on than "350,000".

Broker signals

Metric	Healthy	Problem when
Under-replicated partitions	0	Above 0 for minutes
Offline partitions	0	Above 0
Active controller count	1	0 or changing often
ISR shrinks and expands	Rare	Frequent: slow disks or network
Request latency (produce, fetch)	Low and steady	Rising
Disk used, network utilisation	Well below limits	Near limits

Deciding what to do

1. **Consumers alive?** Check the group state and logs. A crashed consumer or a rebalance loop is more common than a cluster problem.
2. **Processing slow?** Look at the downstream database or API the consumer calls.
3. **Hot partition?** Check key distribution.
4. **Under capacity?** Add consumers up to the partition count; add partitions only with a plan for the key change.
5. **Broker trouble?** Check disks, network and ISR changes.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Lag incident decision path

Part	What it does	What goes wrong
Lag alert	Time behind is above the limit and rising.	A static record-count threshold misleads when traffic changes.
Are consumers alive	Check group state, errors and rebalances.	A rebalance loop looks like lag but is a configuration problem.
Is processing slow	Look at downstream calls and per-record time.	Adding consumers does not help if the database is the limit.
Hot partition or broker	Check key spread, disk and network on brokers.	One broker with a slow disk drags all its partitions.
Scale or fix	Scale consumers, fix code, rebalance keys or add brokers.	Changing partitions mid-incident can break ordering.

Flow: Lag alert to Are consumers alive / Are consumers alive to Is processing slow / Is processing slow to Hot partition or broker / Hot partition or broker to Scale or fix

Capacity and change

Keep disks below about 70 percent, network below 60, and enough headroom to lose one broker and still serve. Roll changes one broker at a time, waiting for under-replicated partitions to reach zero. Test the full restart procedure in a staging cluster.

Incident: The rebalance loop

Incident: *A consumer group rebalanced every few minutes and lag grew, though the cluster was healthy.*

```
consumer: Member c-2 sending LeaveGroup request: consumer poll timeout has expired
coordinator: Preparing to rebalance group invoicer (generation 412)
lag (time behind): 18 minutes and growing
```

Batches had grown and took longer than max.poll.interval.ms, so the coordinator removed the consumer each time. Lowering max.poll.records and raising the interval ended the loop, and the alert now shows minutes behind.

Common mistakes

- Alerting on a fixed lag number
- Adding partitions before checking whether consumers are alive
- Letting disks run above 80 percent
- Restarting brokers quickly without waiting for replicas to catch up

Check yourself

What does lag measure?

Answer: How far a consumer is behind the end of a partition

One partition has huge lag and others none. What is likely?

Answer: A slow consumer or hot key on that partition

What should active controller count be?

Answer: One

Why express lag as time?

Answer: It is easier to understand and alert on

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 10. Managed messaging on the cloud

13 minutes. Compare SQS and SNS, Google Pub/Sub, Azure Service Bus and Event Hubs, and managed Kafka, and choose between running a broker and renting one.

Every major cloud offers messaging as a service. They look similar from far away and differ in ordering, retention, replay and price. Picking from the map of equivalents, then checking the limits that matter to you, keeps both cost and operating work low.

What you will learn

- Queue, topic and stream services on each cloud
- Ordering, retention, replay and delivery per service
- Dead-letter queues and visibility timeouts
- Managed Kafka and when to run your own

The map

Need	AWS	Google Cloud	Azure
Work queue	SQS	Pub/Sub (pull subscription)	Service Bus queue
Fan-out topic	SNS	Pub/Sub topic	Service Bus topic
Partitioned log, replay	Kinesis	Pub/Sub with seek, or Managed Kafka	Event Hubs
Managed Kafka	MSK	Managed Service for Apache Kafka	Event Hubs Kafka endpoint

Things that differ

- **Ordering:** SQS standard is best-effort ordered; FIFO queues order by message group id with lower throughput. Pub/Sub offers ordering keys. Service Bus uses sessions. Event Hubs and Kinesis order within a partition or shard.
- **Delivery:** SQS standard, SNS and Pub/Sub are at-least-once, so consumers must be idempotent. FIFO queues can deduplicate within a window.
- **Retention and replay:** queues delete after handling; logs and Pub/Sub (with retained messages and seek) allow replay for a set time.
- **Failure handling:** use **dead-letter queues** with a maximum receive count, and set the **visibility timeout** (SQS) or acknowledgement deadline longer than processing time, otherwise a message is delivered to a second consumer while the first is still working.
- **Cost:** most services charge per request or per throughput unit; many small messages can cost more than one large batch.

Fan-out pattern

A common design: publish once to a topic (SNS or Pub/Sub), and attach a queue per consumer so each system reads at its own speed with its own dead-letter queue.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Fan-out with managed services

Part	What it does	What goes wrong
Producer service	Publishes an event once to a topic.	Large messages hit size limits; send a pointer to data instead.
Topic	SNS or Pub/Sub topic that copies the event to subscriptions.	A missing subscription means that consumer misses events.
Billing queue	Its own queue and dead-letter queue.	Visibility timeout shorter than the work causes duplicates.
Search queue	Independent speed and retry policy.	An unwatched dead-letter queue hides failures.
Dead-letter queue	Collects messages that exceed the receive count.	Without an alert it silently fills.

Flow: Producer service to Topic / Topic to Billing queue / Topic to Search queue / Billing queue to Dead-letter queue

Run it yourself or rent it

Run Kafka yourself when you need its ecosystem, very high throughput, long replay or low cost at large scale, and you have people to operate it. Otherwise choose the managed service closest to your cloud. Wrap the client in a small interface so a later move is possible.

Incident: Duplicates after a slow job

> **Incident:** *A queue message was delivered twice because processing took longer than the visibility timeout.*

```
$ aws sqs get-queue-attributes --queue-url $Q --attribute-names VisibilityTimeout
VisibilityTimeout: 30
worker log: job 8812 took 95s
worker log: message received 3 times
```

The message became visible again after 30 seconds and another worker picked it up. The team raised the timeout above the longest job, extended it during long jobs, added a dead-letter queue and made the job idempotent.

Common mistakes

- A visibility timeout shorter than the longest job
- No dead-letter queue or no alert on it
- Treating standard queues as ordered
- Choosing self-managed Kafka for a small workload

Check yourself

What does the SQS visibility timeout control?

Answer: How long a received message is hidden from other consumers

Which pattern fans an event out to several queues?

Answer: A topic with a queue per consumer

Which Azure service is a partitioned event log?

Answer: Event Hubs

Why use a dead-letter queue?

Answer: To collect messages that keep failing

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Schemas, contracts and evolving messages

13 minutes. Define message formats with Avro, Protobuf or JSON Schema, use a schema registry, and change formats without breaking producers or consumers.

Messages outlive code. A topic can hold a week or a year of records, and consumers written by other teams read them at different times. A schema and compatibility rules turn a hopeful agreement into something a build can enforce.

What you will learn

- Why a schema beats unstructured JSON
- Backward, forward and full compatibility
- Safe and unsafe changes
- Registry, subjects and CI checks

Schema formats

JSON Schema describes JSON, easy to read, larger on the wire. **Avro** stores data in compact binary and the schema travels by id; it supports defaults and evolution. **Protobuf** uses numbered fields, so names can change but numbers must not.

```
{
  "type": "record", "name": "OrderPaid", "namespace": "shop",
  "fields": [
    {"name": "id", "type": "string"},
    {"name": "total", "type": "double"},
    {"name": "currency", "type": "string", "default": "EUR"}
  ]
}
```

Compatibility

Mode	Meaning	Upgrade order
Backward	New schema can read old data	Consumers first
Forward	Old schema can read new data	Producers first
Full	Both	Any order

Safe changes

- Add a field **with a default** (safe for backward compatibility).
- Remove an **optional** field.
- Never change a field's type, reuse a field name or Protobuf field number, or make an optional field required.
- For a breaking change, **expand then contract**: add the new field, write both, move all consumers, then remove the old one later.

Registry

A **schema registry** stores versions under a **subject** (usually topic name plus key or value). Producers register or look up a schema and send only its id. The registry rejects a new version that breaks the chosen compatibility mode. Run the compatibility check in CI so an incompatible change fails the pull request.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How a schema change is checked

Part	What it does	What goes wrong
Developer changes schema	Adds a field or changes a type in the schema file.	A rename looks harmless but breaks readers.
CI compatibility check	Asks the registry whether the new version is compatible.	Skipping CI lets a breaking change reach production.
Schema registry	Stores versions per subject and enforces the mode.	If it is down, producers may be unable to send.
Producer	Writes records with the schema id.	An old producer keeps writing the old format for a while.
Consumers	Look up the schema by id to decode records.	A consumer on an incompatible schema fails on old records.

Flow: Developer changes schema to CI compatibility check / CI compatibility check to Schema registry / Schema registry to Producer / Producer to Consumers

Incident: Revenue of zero

Incident: *A producer renamed a field and consumers read nulls for a day.*

```
$ psql -c "select sum(amount) from daily_revenue where day = current_date - 1"
0.00
producer: schema v7 renamed amount to total
consumer: schema v6 expects amount, got null
```

The rename was not a compatible change. The team reverted, replayed the day from the topic, and enabled a full-compatibility check in CI. Renames now happen by adding the new field and removing the old one later.

Common mistakes

- Renaming or retyping a field in place
- Required fields with no default
- Reusing a Protobuf field number
- No compatibility check in the build

Check yourself

Which change is safe for backward compatibility?

Answer: Adding a field with a default

What does a schema registry do?

Answer: Stores schema versions and enforces compatibility

What is expand then contract?

Answer: Add the new form, migrate readers, then remove the old

What do Protobuf field numbers require?

Answer: They must never be reused or changed

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. Debugging messaging incidents

14 minutes. Work through a fixed checklist for backlogs, duplicates, lost messages, ordering bugs and stuck consumers across queues and logs.

When a message system misbehaves, the symptom is rarely where the cause is. A queue that grows is a consumer problem; missing data may be a routing or retention problem; duplicates are normal until they are not handled. A checklist per symptom keeps you from changing the wrong thing.

What you will learn

- A checklist for backlog and lag
- A checklist for lost, duplicated and out-of-order messages
- Tracing one message end to end
- What to put in dashboards and alerts

Backlog or lag is growing

1. Are consumers running and connected? (consumer count, group state)
2. Is each consumer making progress? (processing time, errors, rebalance loop)
3. Did the producer rate jump? (publish rate, a retry storm)
4. Is the work stuck on a dependency? (database, API, lock)
5. Is it one partition or queue? (key skew, one broker)

Messages are missing

1. Was the message accepted? (publisher confirm, `acks`, producer errors)
2. Was it routed? (binding, topic and subscription filters, exchange alternate)
3. Is it still retained? (retention time, queue TTL, compaction)
4. Was it acknowledged too early? (auto-ack, commit before processing)
5. Did a failover lose it? (acks=1, non-durable queue, unclean election)

Duplicates and ordering

Duplicates come from redelivery after a crash or timeout, producer retries and rebalances. The cure is an idempotent consumer, not trying to eliminate them. Out-of-order arrival comes from several partitions or queues, concurrent consumers and retries. Cure by key, one consumer per partition, or sequence numbers checked by the consumer.

Trace one message

Every message needs a stable **id** that appears in producer logs, broker headers, consumer logs and the database write. With it, one search shows where the message stopped. Add the id to your trace and log format from the start.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: Following a message

Part	What it does	What goes wrong
Producer log	Logs the message id and confirmation.	Missing confirmations mean it may never have been stored.
Broker metrics	Shows accepted, routed, unroutable and retained counts.	An exchange with no binding drops messages silently.
Consumer log	Logs receipt, success, failure and ack of the id.	No log line for the id means it never arrived.
Database effect	The row or call that proves the work happened.	A row without a log line means the logs are incomplete.
Dead letters	Collects messages that failed repeatedly.	Unwatched dead letters hide the problem.

Flow: Producer log to Broker metrics / Broker metrics to Consumer log / Consumer log to Database effect / Consumer log to Dead letters

Incident: The orders that vanished

Incident: *Some orders never appeared in reports, and nobody could say where they stopped.*

```
producer.log: published order=7731 key=order.created.us
consumer.log: (no entries for order=7731)
broker: exchange events unroutable_dropped 212
```

Orders for one region used a key no binding matched. Following one id showed it stopped at the exchange. The team added a binding, an alternate exchange for unroutable messages and an alert on dropped count.

Common mistakes

- Searching consumer code before checking whether the broker accepted the message
- Logging without a message id
- Treating duplicates as a bug in the broker
- Ignoring the dead-letter queue until a customer complains

Check yourself

A queue grows steadily. What do you check first?

Answer: Whether consumers are running and progressing

Why add a message id to every log line?

Answer: To follow one message through every step

Messages sent but never received, no errors. What might be wrong?

Answer: Routing: no binding or subscription matched

How do you handle duplicates?

Answer: Make consumers idempotent

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Messaging and streaming

RabbitMQ and Kafka commands for queues, topics, consumer groups and lag, with the delivery rules to remember.

Delivery rules

Term	Rule to remember
At-least-once	Duplicates are possible, so handlers must be idempotent
Ordering	Guaranteed only within one queue or partition
Acknowledge	After the work is saved, not before
Dead letters	Every queue needs a failed-message path and an alert on it

RabbitMQ

```
rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
rabbitmqctl list_bindings
rabbitmqctl list_connections name state
rabbitmqadmin declare queue name=billing durable=true
rabbitmqadmin publish exchange=events routing_key=order.paid.eu payload='{"id":4411}'
```

Kafka topics

```
kafka-topics.sh --bootstrap-server b1:9092 --list
kafka-topics.sh --bootstrap-server b1:9092 --describe --topic orders
kafka-topics.sh --bootstrap-server b1:9092 --describe --under-replicated-partitions
kafka-configs.sh --bootstrap-server b1:9092 --entity-type topics --entity-name orders --describe
```

Kafka consumers and lag

```
kafka-consumer-groups.sh --bootstrap-server b1:9092 --list
kafka-consumer-groups.sh --bootstrap-server b1:9092 --describe --group invoicer
kafka-console-consumer.sh --bootstrap-server b1:9092 --topic orders --from-beginning --max-message \
    s 5
```

Producer and consumer settings that matter

Setting	Safe starting value
acks	all
enable.idempotence	true
min.insync.replicas	2 with replication factor 3
enable.auto.commit	false
max.poll.interval.ms	longer than your slowest batch

Debugging order

1. Are consumers running and connected?
2. Is lag on one partition or all of them?
3. Is a dependency slow, or is a message failing in a loop?
4. Is the broker healthy: disk, under-replicated partitions, controller?
5. Trace one message id from producer to database.

Daily checks

```
kafka-consumer-groups.sh --bootstrap-server $B --describe --group orders

kafka-topics.sh --bootstrap-server $B --describe --topic orders

rabbitmqctl list_queues name messages consumers

kafka-console-consumer.sh --bootstrap-server $B --topic orders --from-beginning --max-messages 5
```

MARGA by EknathaLabs marga.eknathalabs.com

Warning: *Resetting consumer group offsets skips or replays messages. Stop the group first and use* `--dry-run` *before* `--execute`*.*

Danger: *Deleting a topic or purging a queue cannot be undone. Check the name twice and confirm the environment.*

Warning: *Auto-acknowledge loses messages when a consumer crashes mid-work. Acknowledge after the work is stored.*

Warning: *Lowering retention deletes old data on the next cleanup. Check what slow consumers still need first.*

MARGA by EknathaLabs
marga.eknathalabs.com